

Rome Wasn't Built in a Day — But It Was Built to a Plan

Every implementation has a plan on day one. Few still resemble that plan by go-live — not because the plan was wrong, but because nobody was tracking the thing that actually mattered. Here's how to track toward the outcome, absorb real change without losing the thread, and still ship on a timeline.

11 MIN READ · SUPPLY CHAIN CAPABILITIES

IN THIS DOCUMENT

1. Half the battle, not the whole war
2. What "on track" actually means
3. The whack-a-mole problem
4. Being agile without losing the plan
5. What to let go, and when
6. UAT as proof, not a checklist
7. The tracking framework
8. How we actually run this

Half the battle, not the whole war

"Half the battle is won in discovery" is true, and it's also where most teams stop applying it too early. A requirements document tells you *what* the business asked for. It rarely tells you *why* — and the why is what you need eleven weeks later, when a requirement turns out to be wrong, incomplete, or in conflict with another one.

Take a scenario that plays out on almost every SCM implementation. The warehouse team asks for a new field on the receiving screen: "Quality Hold Reason." That's the requirement, and it's easy to build exactly as stated — one text field, done in an afternoon. What nobody captured is the outcome behind it: the quality team was taking four days to disposition a hold because nobody told them one existed until someone happened to notice it sitting in the yard. Built literally, the field is just a place to type a note nobody reads. Built against the outcome — cut disposition time on quality holds — that same field also needs to fire a notification to the quality queue the moment it's entered. Same requirement, same afternoon of effort either way. Only one version actually fixes anything.

NUGGET

In every discovery workshop, capture one extra column next to each requirement: "what business result does this enable?" It costs almost nothing to ask, and it's the single sentence that lets you make good trade-off calls later without going back to the business every time.

What "on track" actually means

Most status reports track tasks: percent complete, stories closed, tickets resolved. None of that tells you whether the business is actually going to get what it needed. A project can be 90% done on tasks and still fail the thing it was funded to achieve, because the tasks were never re-anchored to the outcome once things started changing.

SIGNAL	TASK-BASED TRACKING	OUTCOME-BASED TRACKING
What it measures	Work completed	Business capability enabled

Typical status	"72% of stories closed"	"Order-to-cash cycle time target is on pace / at risk"
Where it hides risk	Doesn't surface scope drift	Surfaces drift immediately
Who it's useful to	The delivery team, internally	The business sponsor, and the delivery team

Both are needed — you still have to know what's built. But the milestone review has to answer the outcome question first, and the task list second. Reverse that order and you get teams that report green for months and then discover, at UAT, that the outcome was never actually on track.

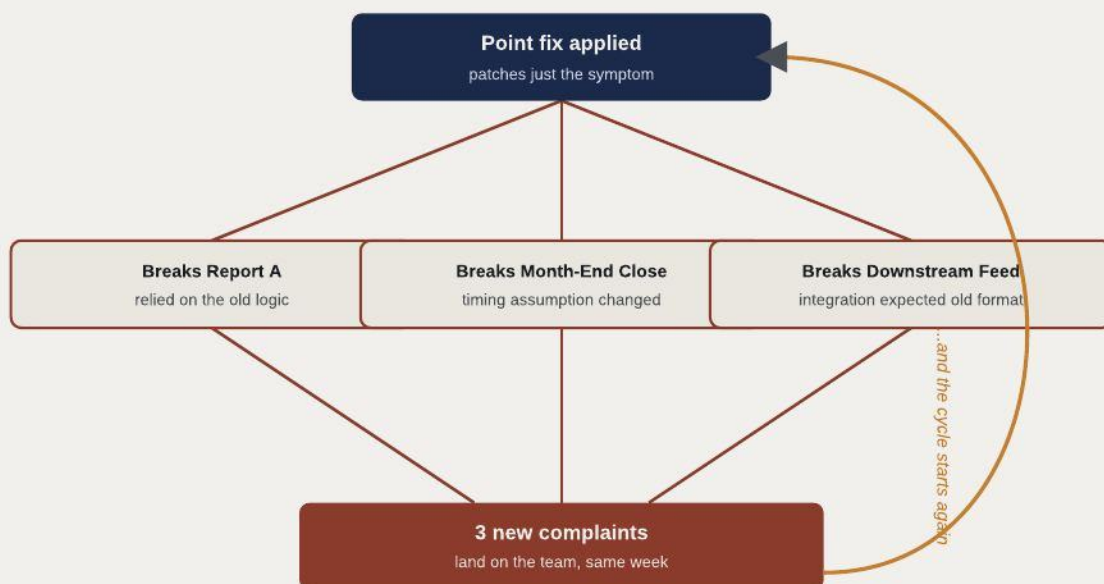
The whack-a-mole problem

This is the failure mode behind the question you're really asking: why does fixing one thing so often break three others? Because the fix was scoped to the complaint, not to the system it lives in. Someone reports that inventory counts look wrong on one report, and the fastest fix is a patch to that report — which quietly breaks the three other places that relied on the same logic behaving the way it used to.

CAVEAT

A team that spends its sprint capacity reacting to the loudest issue is not implementing a business process — it's running a help desk with extra steps. Every fix needs to be evaluated against the process it sits inside, not just the symptom that got reported.

Left unchecked, this doesn't stay a one-time mistake — it becomes a loop the team can't climb out of. One point fix breaks three things downstream; fixing those three, the same way, breaks more; and the team spends the rest of the milestone patching its own patches instead of building anything new.



STUCK IN THE VICIOUS CIRCLE

Three patches, spent fixing what the first patch broke — and nothing built for the business process it was supposed to serve.

This is precisely why the outcome — the "what business result does this enable" column from discovery — matters mid-project. It's the thing that tells you whether a fix is a real fix or a relocation of the same problem. Without it, every fix is a local decision made by whoever is looking at the screen that week, and the system slowly becomes a patchwork with no one holding the whole map.

Being agile without losing the plan

Requirements will change mid-implementation. That's not a failure of discovery — it's what happens when a business actually sees the system taking shape and realizes something they didn't know to ask for in week two. The mistake isn't that requirements change; it's treating every change as either an emergency that jumps the queue, or a threat to be resisted on principle. Both reactions lose the plan.

Agile, done properly here, isn't "no plan" — it's a plan with a deliberate absorption mechanism built in: a backlog that's re-prioritized on a fixed cadence, not whenever someone asks loudly; a change log that records what changed, why, and what it displaced; and a milestone that's protected even when its contents shift underneath it.

TIP

Protect the milestone date, not the original scope list. When a genuine change lands, ask "what comes out to make room for this" in the same conversation as "should we do this" — not as a separate negotiation two weeks later.

What to let go, and when

Not every good idea belongs in this milestone. Some of the best implementation decisions are the ones where a genuinely valid requirement gets deliberately parked for a later phase — not dropped, not forgotten, but sequenced. All roads can still lead to Rome without every road being built in year one.

The test isn't "is this a good idea" — almost everything raised in a workshop is a good idea to someone. The test is "does the core business outcome for this milestone survive without it." If yes, it goes in the parking lot with a named owner and a target phase. If no, it's not an addition — it was always part of the milestone, and the plan was wrong to leave it out.

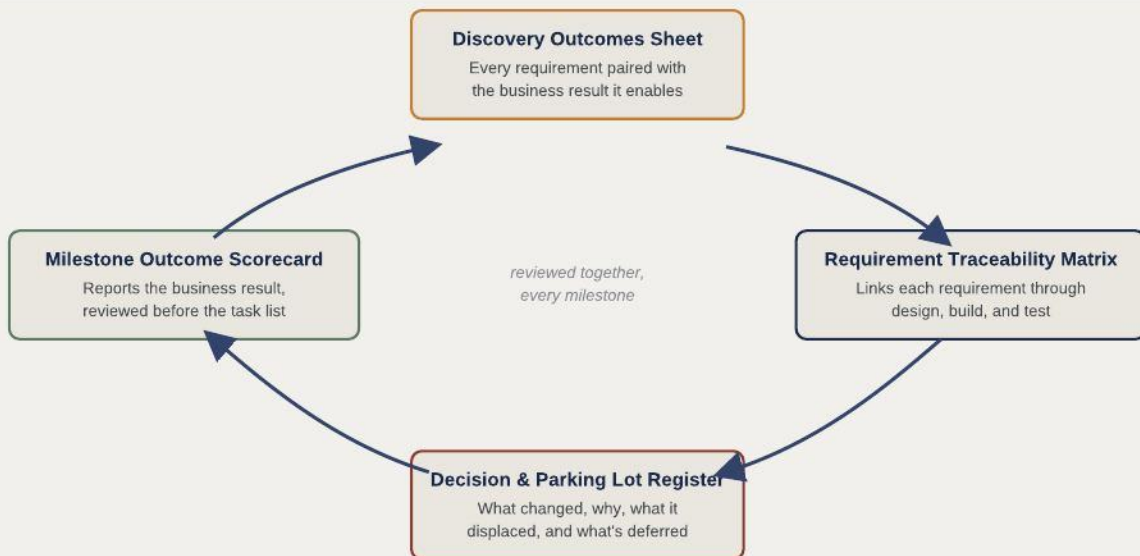
UAT as proof, not a checklist

UAT scripted purely against functional steps — click here, enter this, confirm that field populates — proves the system does what the build team built. It doesn't prove the system does what the business needed, because those two things quietly drift apart over months of change requests.

The fix is the same discovery column, used one more time: every UAT scenario should trace back to a named business outcome, not just a requirement ID. When a tester validates a scenario, they're validating that the outcome is achievable — which catches the case where every individual requirement was technically met, but the business process they were supposed to add up to still doesn't work end to end.

The tracking framework

None of this works as an idea you keep in your head. It has to live in four connected artifacts, reviewed on the same cadence as your milestones — not four separate tools nobody opens.



The four artifacts feed each other in a loop — one without the others just produces more documents nobody reconciles.

The point of the loop isn't the paperwork. It's that a requirement change, a UAT failure, and a scope-deferral decision all get evaluated against the same outcome statement — so "should we do this" and "does this still get us to Rome" are always the same question.

INSU DYNAMICS

How we actually run this

This isn't a framework we describe in a proposal and then abandon at kickoff. It's how our delivery teams are structured to work, on every engagement.

Discovery captures requirements but not the "why"

Our discovery template requires a named business outcome for every requirement before it's accepted into scope — no outcome, no ticket.

Status reporting hides scope drift behind task percentages

Our milestone reviews open with the Outcome Scorecard, not the task burndown — the business sponsor sees outcome risk before anyone sees velocity.

Fixes patch symptoms and break adjacent processes

Every fix is checked against the Requirement Traceability Matrix for what else depends on the logic being touched, before it's built — not after it breaks.

Requirement changes either jump the queue or get blocked on principle

We run a fixed-cadence backlog review where every change is weighed against what it displaces, logged in the Decision Register with the trade-off made explicit.

Good ideas get crammed into a milestone that can't hold them

We maintain a live Parking Lot with a named owner and target phase per item, visible to the business — deferred is not the same as dropped, and we can show that.

UAT proves the build, not the business process

Every UAT scenario we write traces back to a discovery outcome, not just a requirement ID — so sign-off means the process works, not just that the screen does.

This is what **innovation drives sustainability** means for us in practice — not a slogan, but a tracking discipline that gets the business to the outcome it actually asked for, on a timeline it can plan around.